Studio Nx

P1-16 대시보드 API — 함수 설명 합본 (63~69)

2026-10-12

목차

1. ai-notes/backend(fastAPI_Python)/P1-16 - 대시보드 API/63_model_project_session.md
2. ai-notes/backend(fastAPI_Python)/P1-16 - 대시보드 API/64_dashboard_get_storage.md
3. ai-notes/backend(fastAPI_Python)/P1-16 - 대시보드 API/65_dashboard_get_ai_usage.md
4. ai-notes/backend(fastAPI_Python)/P1-16 - 대시보드 API/66_dashboard_get_recent_projects.md
5. ai-notes/backend(fastAPI_Python)/P1-16 - 대시보드 API/67_dashboard_get_ready_projects.md
6. ai-notes/backend(fastAPI_Python)/P1-16 - 대시보드 API/68_dashboard_get_dashboard.md
7. ai-notes/backend(fastAPI_Python)/P1-16 - 대시보드 API/69_me_route_dashboard.md

Backend(FastAPI/Python) 63 — `ProjectSession` 나머지 컬럼 (project_sessions)

항목	내용
파일	`studionx/backend/app/models/project_session.py`
작업	P1-16 대시보드 API: 작업 세션 모델 완성 (PRD 01 FR-128, FR-130, DB.md §3.1)
작성	Claude (AI), 2026-10-12, 사용자 검토
이전 · 다음	62 프로젝트 라우터 · 64 `get_storage()`

1. 하는 일

`project_sessions` 표는 시퀀서를 연 기록을 담는 표입니다. 이 표의 ORM 모델에 **아직 없던 칸 3개**를 추가합니다. 표 자체는 이미 DB에 있었고(`studionx/db/01_phase1_account.sql`), 모델 쪽만 템플릿 TODO로 남아 있었습니다.

칸	타입 (DB)	기본값	뜻
`started_at`	`DATETIME(3)` NOT NULL	지금 시각 (DB가 채움)	시퀀서에서 프로젝트를 연 시각 (UTC)
`last_seen_at`	`DATETIME(3)` NOT NULL	지금 시각 (DB가 채움)	마지막 신호 시각: 시퀀서가 1분마다 갱신합니다 (Phase 2)
`active_seconds`	`INT UNSIGNED` NOT NULL	0	5분 넘게 입력이 없던 구간을 뺀 실제 작업 시간 (초)

예를 들어 한국 시각 오후 3시에 시퀀서를 열어 3시 45분까지 작업했고, 중간에 5분 넘게 쉰 적이 없다면 `started_at`은 `06:00:00`(UTC), `last_seen_at`은 `06:45:00`, `active_seconds`는 `2700`이 됩니다.

순서를 이렇게 정한 이유: 다음에 만들 함수들(최근 프로젝트, 작업 시간, Ready to start)이 이 칸들을 읽습니다. 모델에 칸이 없으면 `ProjectSession.started_at` 같은 조건을 코드로 쓸 수 없기 때문에, 모델을 먼저 완성했습니다.

2. 언제 쓰나

쓰는 곳	하는 일
66 `get_recent_projects` (예정)	`started_at`이 최근 7일 안인 내 세션을 읽어 마이페이지 작업 시간 그래프에 씁니다 (FR-128)
67 `get_ready_projects` (예정)	내 세션이 하나도 없는 프로젝트를 찾아 Ready to start 배너에 씁니다 (FR-130)
P1-14 `to_item`, `list_projects`	`opened` 값은 이미 `session_id`, `project_id`, `user_id`만으로 계산하고 있어서 영향이 없습니다
Phase 2 시퀀서 (FR-257)	시퀀서를 열 때 행을 만들고, 1분마다 `last_seen_at`과 `active_seconds`를 갱신합니다

3. 코드

import (파일 맨 위)

```
from datetime import datetime

from sqlalchemy import BigInteger, ForeignKey, text
from sqlalchemy.dialects.mysql import DATETIME, INTEGER
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`datetime`, `text`, `DATETIME`, `INTEGER`를 이번에 추가했습니다. 세 칸의 타입과 DB 기본값을 정하는 데 씁니다.

클래스 (추가한 부분)

```
    # (구현: AI — 2026-10-12, P1-16) 시각은 UTC · 시간대 없이 저장 (다른 시각 칸과 같음)
    started_at: Mapped[datetime] = mapped_column(  # 시퀀서에서 연 시각
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    last_seen_at: Mapped[datetime] = mapped_column(  # 마지막 신호 (1분마다, Phase 2)
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    active_seconds: Mapped[int] = mapped_column(  # 5분 넘게 입력 없는 구간은 뺀 작업 시간
        INTEGER(unsigned=True), server_default=text("0")
    )
```

문법 풀이

import 문법	뜻
`from sqlalchemy.dialects.mysql import DATETIME, INTEGER`	MySQL 전용 타입입니다. 소수 자리(`fsp`)와 부호 없음(`unsigned`)을 정할 수 있습니다
`from sqlalchemy import text`	SQL 문장을 그대로 넘깁니다. 기본값 `CURRENT_TIMESTAMP(3)`을 DB가 채우게 할 때 씁니다

식	뜻	예
`Mapped[datetime]`	파이썬에서 이 칸은 `datetime`이고, `| None`이 없으므로 NULL을 허용하지 않습니다	
`DATETIME(fsp=3)`	소수 셋째 자리(밀리초)까지 저장합니다	`2026-10-10 03:00:00.123`
`INTEGER(unsigned=True)`	0 이상의 정수만 받습니다. 음수를 넣으면 DB 오류가 납니다	`-1` → 오류
`server_default=text("…")`	값을 넣지 않으면 파이썬이 아니라 **DB가** 기본값을 채웁니다	지금 시각, 0

4. 흐름

모델에 칸이 생겼을 뿐, 새로 동작하는 기능은 없습니다. 행을 넣을 때 시각과 작업 시간을 주지 않으면 DB가 지금 시각과 0으로 채우고, 읽을 때는 시간대 정보가 없는 `datetime`(UTC 값)으로 돌아옵니다.

5. 왜 이렇게

선택	이유
SQL 정의와 **똑같이** 맞춤 (`DATETIME(3)`, `INT UNSIGNED`, 기본값)	모델과 DB가 어긋나면 시험과 실제 운영에서 다르게 동작합니다. 그래서 시험에서 칸 이름, 순서, NULL 여부, FK를 실제 DB와 비교합니다
기본값은 DB가 채움 (`server_default`)	`projects.created_at` 같은 다른 시각 칸과 같은 방식입니다. 서버 시계가 아니라 DB 시계 하나를 기준으로 삼습니다
UTC로, 시간대 정보 없이 저장	다른 시각 칸과 같은 규칙입니다. 요일이나 "오늘"을 나누는 일은 화면이 브라우저 시간대로 합니다 (DB.md §3.1)
`active_seconds`를 따로 둠	`last_seen_at - started_at`으로 계산하면 창을 켜 둔 채 쉰 시간까지 포함되어 실제 작업 시간과 달라집니다

6. 시험 (`tests/test_project_session_model.py` 새 파일, 4개)

시험	확인한 것
컬럼 비교	6칸의 이름, 순서, NULL 불가, FK 2개(`project_id`는 ON DELETE CASCADE)가 실제 MySQL과 같습니다
기본값	칸을 비우고 넣으면 두 시각이 시간대 정보 없이 채워지고, `active_seconds`는 0입니다
값 보존	`03:00:00.123`의 밀리초와 2400초가 그대로 저장됩니다
음수	`active_seconds = -1`을 넣으면 DB 오류가 납니다 (UNSIGNED)

전체 시험 496개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 64 — `get_storage()` (저장 용량)

항목	내용
파일	`studionx/backend/app/services/dashboard_service.py`
작업	P1-16 대시보드 API: 마이페이지의 용량 칸 (PRD 01 FR-131, FR-133)
작성	Claude (AI), 2026-10-12, 사용자 검토
이전 · 다음	63 `ProjectSession` 컬럼 · 65 `get_ai_usage()`

1. 하는 일

이 함수는 회원의 **저장 용량 한도와 사용량**을 마이페이지 응답 모양(`DashboardStorage`)으로 돌려줍니다.

회원	결과
새 회원 (DB 기본값)	`{"quota_bytes": 5368709120, "used_bytes": 0, "by_kind": {"samples": 0, "exports": 0, "stems": 0, "snapshots": 0}}`: 5GB 중 0을 쓴 상태입니다
한도가 10GB이고 1.23GB를 쓴 회원 (관리자가 한도를 바꾼 경우)	`quota_bytes`는 10737418240, `used_bytes`는 1234567890입니다
프로필 사진이 있는 회원	`used_bytes`는 그대로 0입니다. 사진은 사용량에 넣지 않습니다

2. 언제 쓰나

호출하는 곳	화면
68 `get_dashboard`(예정) → `GET /me/dashboard`	myPage.html의 용량 막대 "0 B of 5 GB" (P1-17)

용량 숫자의 출처: 한도는 `users.storage_quota_bytes`(기본 5GB), 사용량은 `users.storage_used_bytes`(기본 0)에서 가져옵니다. 파일을 올리고 지울 때 사용량을 늘리고 줄이는 일은 샘플, 녹음, 출력 기능이 생기는 **Phase 2**에서 합니다. 그래서 지금은 모든 회원의 사용량이 0입니다.

3. 코드

import (파일 맨 위)

```
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.dashboard import (
    DashboardAi,
    DashboardResponse,
    DashboardStorage,
    ReadyProject,
    RecentProject,
    StorageByKind,
)
```

`StorageByKind`를 이번에 추가했습니다. 이 함수는 `User`, `DashboardStorage`, `StorageByKind`를 사용합니다.

함수

```
def get_storage(user: User) -> DashboardStorage:
    return DashboardStorage(
        quota_bytes=user.storage_quota_bytes,
        used_bytes=user.storage_used_bytes,
        by_kind=StorageByKind(),
    )
```

문법 풀이

import 문법	뜻
`from app.schemas.dashboard import StorageByKind`	종류별 사용량의 응답 모양입니다. 네 칸 모두 기본값이 0입니다

식	뜻	예
`StorageByKind()`	값을 하나도 넘기지 않으면 모든 칸이 기본값(0)이 됩니다	`{samples: 0, exports: 0, stems: 0, snapshots: 0}`
`-> DashboardStorage`	돌려주는 값의 모양입니다. FastAPI가 이것을 JSON으로 바꿉니다	
`db` 매개변수가 없음	DB를 다시 읽지 않습니다. `user`는 토큰 검사(P1-03 `get_current_user`)가 이미 읽어 온 행입니다	

4. 흐름

회원 행에 있는 두 숫자와, 0으로 채운 종류별 사용량을 묶어서 돌려줍니다.

5. 왜 이렇게

선택	이유
`users` 칸을 그대로 씀	요청할 때마다 파일 표에서 크기를 더하면 느립니다. 그래서 파일을 올리고 지울 때 미리 갱신해 두는 방식을 씁니다 (FR-133, DB.md)
종류별 사용량은 0	Phase 1에는 samples나 exports 같은 표가 없습니다 (템플릿). 그래도 칸은 지금부터 내보내서, 화면(P1-17)이 처음부터 같은 모양을 쓰게 했습니다
프로필 사진은 제외	회원당 2MB 한 장뿐이라 "작업 파일 용량"에 넣지 않기로 했습니다 (2026-10-07 결정)
DB를 다시 읽지 않음	같은 요청 안에서 이미 읽어 온 회원 행이므로, 쿼리를 아낄 수 있습니다

6. 시험 (`tests/test_dashboard_service.py` 새 파일, 4개)

시험	확인한 것
기본값	새 회원은 한도 5GB(5368709120), 사용량 0, 종류별 사용량이 모두 0입니다
회원 값	한도 10GB와 사용량 12345678900이 그대로 나옵니다
프로필 사진	사진이 있어도 사용량은 0입니다

시험	확인한 것
JSON 모양	칸이 3개이고, 한도는 정수입니다

전체 시험 500개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 65 — `get_ai_usage()` (오늘 AI 사용량)

항목	내용
파일	`studionx/backend/app/services/dashboard_service.py`
작업	P1-16 대시보드 API: 마이페이지의 AI 사용량 칸 (PRD 01 FR-136, PRD 05 FR-551, FR-553)
작성	Claude (AI), 2026-10-12, 사용자 검토
이전 · 다음	64 `get_storage()` · 66 `get_recent_projects()`

1. 하는 일

이 함수는 **오늘** 사용한 AI 횟수와 하루 한도를 돌려줍니다. Phase 1에는 AI 기능이 없으므로 사용한 횟수는 항상 0입니다.

칸	값	뜻
`analyses`	`{"used": 0, "limit": 5}`	레퍼런스 트랙 분석: 하루 5번까지 (PRD 05)
`questions`	`{"used": 0, "limit": 30}`	Nyxi에게 질문하기: 하루 30번까지

마이페이지(P1-17)에는 "AI analyses 0 / 5 · Questions 0 / 30"과 같은 형태로 표시됩니다.

2. 언제 쓰나

호출하는 곳	화면
68 `get_dashboard`(예정) → `GET /me/dashboard`	myPage.html의 AI 사용량 칸 (P1-17)

"오늘"의 기준은 UTC 0시입니다 (2026-10-12 사용자 결정 A). 그래서 사용량은 한국 시각으로 **매일 오전 9시**에 0으로 돌아갑니다.

선택	하루가 바뀌는 시각	결정
A. UTC 0시	모든 회원이 같은 시각 (한국 오전 9시)	✅ 채택했습니다. 회원의 시간대 정보가 필요하지 않습니다
B. 사용자 시간대 0시	각자의 자정	Phase 5에서 필요해지면 도입합니다. 회원별 시간대를 저장할 칸이 있어야 하는데, 지금 DB에는 없습니다

3. 코드

import와 상수 (파일 맨 위)

```
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.dashboard import (
    DashboardAi,
```

```
    DashboardResponse,
    DashboardStorage,
    ReadyProject,
    RecentProject,
    StorageByKind,
    UsageCount,
)

# AI 일일 한도 (PRD 05 FR-551) — 지금은 고정값, 나중에 core/config.py 설정으로 옮기기
AI_DAILY_ANALYSES = 5
AI_DAILY_QUESTIONS = 30
```

`UsageCount`를 이번에 추가했습니다. 이 함수는 `Session`, `User`, `DashboardAi`, `UsageCount`와 두 상수를 사용합니다.

함수

```
def get_ai_usage(db: Session, user: User) -> DashboardAi:
    return DashboardAi(
        analyses=UsageCount(used=0, limit=AI_DAILY_ANALYSES),
        questions=UsageCount(used=0, limit=AI_DAILY_QUESTIONS),
    )
```

Phase 5에서 바뀔 모양 (참고용이며, 지금 코드에는 없습니다)

```
today = datetime.now(UTC).replace(hour=0, minute=0, second=0, microsecond=0, tzinfo=None)
used = db.scalar(select(func.count()).select_from(AiAnalysis)
                 .where(AiAnalysis.user_id == user.user_id, AiAnalysis.created_at >= today))
```

문법 풀이

import 문법	뜻
`from app.schemas.dashboard import UsageCount`	`{used, limit}` 두 칸으로 된 응답 모양입니다

식	뜻	예
`UsageCount(used=0, limit=5)`	칸 이름을 지정해서 값을 넣어 만듭니다	`{"used": 0, "limit": 5}`
`db`와 `user`를 받지만 쓰지 않음	Phase 5에서 횟수를 셀 때 필요합니다. 이 함수를 부르는 쪽(68)을 그때 고치지 않아도 되도록 지금부터 같은 모양으로 둡니다	
`.replace(hour=0, …)` (Phase 5)	지금 UTC 시각의 시, 분, 초를 0으로 바꿔서 오늘 UTC 0시를 만듭니다	

4. 흐름

두 한도 상수와 0으로 응답을 만들어 돌려줍니다.

5. 왜 이렇게

선택	이유
지금은 사용량이 0	Phase 1에는 AI 기능도, 사용 기록을 남길 표도 없습니다 (템플릿). 그래도 칸은 지금부터 내보내서 화면이 처음부터 같은 모양을 쓰게 했습니다
UTC 0시 (A)	사용자가 결정했습니다. 다른 시각 칸도 모두 UTC로 저장하므로 비교가 단순하고, 회원의 시간대 정보도 필요하지 않습니다
매개변수를 미리 받아 둠	템플릿의 모양을 그대로 따랐습니다. Phase 5에서는 함수 안쪽만 바꾸면 됩니다
한도는 상수로 둠	PRD 05에 정해진 값입니다. 나중에 `core/config.py` 설정으로 옮겨서 서버마다 바꿀 수 있게 할 예정입니다 (템플릿 메모)

6. 시험 (`tests/test_dashboard_service.py`: 3개를 추가해서 모두 7개)

시험	확인한 것
한도 값	5와 30입니다 (PRD 05)
Phase 1 응답	`{"analyses": {"used": 0, "limit": 5}, "questions": {"used": 0, "limit": 30}}`
DB를 읽지 않음	`db` 자리에 `None`을 넘겨도 동작합니다. 지금은 쿼리가 한 번도 나가지 않습니다

전체 시험 503개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 66 — `get_recent_projects()` (최근 프로젝트 · 작업 시간)

항목	내용
파일	`studionx/backend/app/services/dashboard_service.py`
작업	P1-16 대시보드 API: 마이페이지의 최근 프로젝트 5개와 최근 7일 작업 세션 (PRD 01 FR-121, FR-128)
작성	Claude (AI), 2026-10-12, 사용자 검토
이전 · 다음	65 `get_ai_usage()` · 67 `get_ready_projects()`

1. 하는 일

이 함수는 내가 참여한 프로젝트 중 **최근에 수정한 5개**와, 각 프로젝트에서 **최근 7일 동안 내가 시퀀서를 연 기록(세션)**을 돌려줍니다. 마이페이지(P1-17)는 이 데이터로 "최근 프로젝트" 목록과 요일별 작업 시간 막대를 그립니다.

데이터	결과
내 프로젝트 7개, 삭제한 프로젝트 1개, 남의 프로젝트 1개, 멤버로 참여한 남의 프로젝트 1개	내 것과 멤버로 참여한 것 중 삭제되지 않은 프로젝트를 최근에 수정한 순서로 **5개** 돌려줍니다
프로젝트 A에 내 세션이 1일 전, 3일 전, 8일 전에 있고 다른 사람의 세션도 있음	A의 `sessions`에는 **3일 전 → 1일 전** 세션 두 개만 들어갑니다 (8일 전 세션과 다른 사람의 세션은 빠집니다)
프로젝트가 없음	`[]`를 돌려줍니다

응답 한 줄의 예:

```
{"project_id": 12, "title": "Night Drive", "bpm": "128.50", "music_key": "A minor",
 "updated_at": "2026-10-12T02:10:11.123000",
 "sessions": [{"started_at": "2026-10-09T03:00:00", "last_seen_at": "2026-10-09T03:30:00", "active_seconds": 300}]}
```

2. 언제 쓰나

호출하는 곳	화면
68 `get_dashboard`(예정) → `GET /me/dashboard`	myPage.html의 Recent projects와 이번 주 작업 시간 (P1-17)

요일은 화면이 나눕니다. 서버는 세션 시각을 **UTC 그대로** 보내고, 화면이 브라우저 시간대에 맞춰 "월, 화, 수 …"로 나눠 더합니다 (DB.md §3.1). 그래서 서버는 사용자의 시간대를 몰라도 됩니다. 여기서 "7일"은 지금으로부터 7 × 24시간을 뜻합니다 (`started_at >= 지금 - 7일`).

Phase 1에는 세션을 만드는 기능(시퀀서, Phase 2)이 없어서 `sessions`는 비어 있습니다. 시험에서는 세션 행을 직접 넣어서 확인했습니다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.models import Project, ProjectMember, ProjectSession, User
from app.schemas.dashboard import (
    DashboardAi,
    DashboardResponse,
    DashboardStorage,
    ReadyProject,
    RecentProject,
    SessionItem,
    StorageByKind,
    UsageCount,
)
```

`datetime` 줄과 `select`, `Project`, `ProjectMember`, `ProjectSession`, `SessionItem`을 이번에 추가했습니다. 이 함수는 상수 `RECENT_PROJECT_COUNT`(5)와 `RECENT_DAYS`(7)도 사용합니다.

함수

```
def get_recent_projects(db: Session, user: User) -> list[RecentProject]:
    projects = db.scalars(
        select(Project)
        .join(ProjectMember, ProjectMember.project_id == Project.project_id)
        .where(ProjectMember.user_id == user.user_id, Project.deleted_at.is_(None))
        .order_by(Project.updated_at.desc(), Project.project_id.desc())
        .limit(RECENT_PROJECT_COUNT)
    ).all()
    if not projects:
        return []

    since = datetime.now(UTC).replace(tzinfo=None) - timedelta(days=RECENT_DAYS)
    rows = db.scalars(
        select(ProjectSession)
        .where(
            ProjectSession.project_id.in_([p.project_id for p in projects]),
            ProjectSession.user_id == user.user_id,
            ProjectSession.started_at >= since,
        )
        .order_by(ProjectSession.started_at)
    ).all()
    sessions = {p.project_id: [] for p in projects}
    for row in rows:
        sessions[row.project_id].append(
            SessionItem(
                started_at=row.started_at,
                last_seen_at=row.last_seen_at,
                active_seconds=row.active_seconds,
            )
```

```
        )

    return [
        RecentProject(
            project_id=p.project_id,
            title=p.title,
            bpm=p.bpm,
            music_key=p.music_key,
            updated_at=p.updated_at,
            sessions=sessions[p.project_id],
        )
        for p in projects
    ]
```

문법 풀이

import 문법	뜻
`from datetime import UTC, datetime, timedelta`	지금 시각(UTC)과 기간("7일")을 다룹니다
`from sqlalchemy import select`	SELECT 문을 만듭니다
`from app.schemas.dashboard import SessionItem`	세션 한 개의 응답 모양입니다 (시작 시각, 마지막 신호, 작업 시간 초)

식	뜻	예
`db.scalars(select(Project)…).all()`	행마다 `Project` 하나씩 담긴 목록을 가져옵니다	
`.join(ProjectMember, …).where(ProjectMember.user_id == 나)`	내가 멤버인 프로젝트만 고릅니다 (OWNER가 아니어도 됩니다)	
`.limit(RECENT_PROJECT_COUNT)`	5개만 가져옵니다	
`datetime.now(UTC).replace(tzinfo=None) - timedelta(days=7)`	지금(UTC)으로부터 7일 전 시각입니다. DB와 같은 "시간대 정보가 없는 UTC" 형식입니다	
`.in_([p.project_id for p in projects])`	`project_id IN (12, 9, 7, …)`: 다섯 프로젝트를 한 번에 조회합니다	
`{p.project_id: [] for p in projects}`	프로젝트 번호마다 빈 목록을 만듭니다. 그래서 세션이 없는 프로젝트도 `[]`가 됩니다	`{12: [], 9: []}`
`sessions[row.project_id].append(…)`	해당 프로젝트의 목록에 세션을 추가합니다	
`[RecentProject(…) for p in projects]`	프로젝트 순서(최근 수정순)를 그대로 유지해서 응답 목록을 만듭니다	

4. 흐름

1. 내가 멤버이고 삭제되지 않은 프로젝트를 최근에 수정한 순서로 5개 가져옵니다. 하나도 없으면 `[]`를 돌려줍니다.
2. 그 5개 프로젝트에 있는 **내** 세션 중 최근 7일 안의 것을 **한 번에** 시작 시각 순서로 가져옵니다.
3. 세션을 프로젝트별로 나눠 담고, 프로젝트 순서대로 응답을 만듭니다.

5. 왜 이렇게

선택	이유
세션을 쿼리 **한 번**으로 조회	프로젝트마다 따로 물으면 5번이 나갑니다 (N+1 문제). `IN (...)`으로 한 번에 조회했고, 시험에서 SELECT가 **2번**(프로젝트, 세션)만 나가는 것을 확인했습니다 (템플릿 권장 방식)
내 세션만 셈	마이페이지에 보여 줄 것은 "내 작업 시간"이기 때문입니다 (FR-128). 같은 프로젝트의 다른 멤버 기록은 빼야 합니다
수정 시각이 같으면 `project_id`가 큰 것을 먼저	58번 `list_projects`와 같은 규칙입니다. 두 화면의 순서가 같아집니다
세션은 오래된 것부터 정렬	화면이 시간 순서대로 막대를 쌓기 쉽습니다
UTC 그대로 보내고 요일은 화면이 나눔	서버가 사용자의 시간대를 몰라도 됩니다. 65번의 "오늘" 결정과 같은 방향입니다
프로젝트가 없으면 바로 `[]`	빈 `IN ()` 조건으로 쿼리를 보내지 않기 위해서입니다

6. 시험 (`tests/test_dashboard_service.py`: 5개를 추가해서 모두 12개)

시험	확인한 것
프로젝트가 없음	`[]`를 돌려줍니다
5개와 범위	내 것 7개, 삭제한 것, 남의 것, 멤버로 참여한 남의 것을 넣으면 `Mine 0, Shared, Mine 1, Mine 2, Mine 3`이 최근 수정순으로 5개 나옵니다
응답 칸	bpm `128.50`, 키 `A minor`가 들어 있고, 세션이 없으면 `[]`입니다
세션 범위와 순서	A에서는 내 1일 전, 3일 전 세션이 `[300, 100]`(오래된 것 먼저)으로 나오고, 8일 전 세션과 다른 사람의 세션은 빠집니다. B에서는 6.9일 전 세션이 포함됩니다
쿼리 수	프로젝트 5개와 세션 5개가 있어도 SELECT는 **2번**만 나갑니다

전체 시험 508개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 67 — `get_ready_projects()` (아직 열지 않은 프로젝트)

항목	내용
파일	`studionx/backend/app/services/dashboard_service.py`
작업	P1-16 대시보드 API: 마이페이지의 'Ready to start' 배너 (PRD 01 FR-130)
작성	Claude (AI), 2026-10-12, 사용자 검토
이전 · 다음	66 `get_recent_projects()` · 68 (다음)

1. 하는 일

이 함수는 **내가 시퀀서에서 아직 한 번도 열지 않은** 프로젝트를 최근에 만든 순서대로 최대 3개까지 돌려줍니다. 마이페이지(P1-17) 맨 위에 있는 **'Ready to start'** 배너는 그중 첫 번째 프로젝트를 보여 주고, 나머지 개수는 "1 more in My Projects"처럼 알려 줍니다.

데이터	결과
내가 30일 전에 한 번 열었던 프로젝트	빠집니다 (한 번이라도 열었으면 제외)
새로 만든 프로젝트 4개	최근에 만든 순서로 나옵니다
삭제한 프로젝트, 내가 멤버가 아닌 남의 프로젝트	빠집니다
멤버로 참여한 남의 프로젝트 중 **그 사람은 열었지만 나는 열지 않은 것**	포함됩니다
위 데이터를 모두 합친 경우	`New 0, Shared, New 1` (최대 3개)

작은 화면에서 프로젝트를 만든 사람이 나중에 컴퓨터로 접속하면, 이 배너가 그 프로젝트를 곧바로 보여 줍니다. 이것은 FR-129와 FR-130에 정해진 동작이며, 36번 안내 모달의 문구 "it'll be waiting at the top of your dashboard"가 가리키는 기능이기도 합니다.

2. 언제 쓰나

호출하는 곳	화면
68 `get_dashboard`(예정) → `GET /me/dashboard`	myPage.html의 'Ready to start' 배너 (P1-17)

`NOT EXISTS`(서브쿼리): "이 조건에 맞는 행이 **하나도 없으면**"이라는 뜻을 SQL로 표현하는 방법입니다. 프로젝트마다 "내 세션 행이 있는가?"를 DB 안에서 확인하기 때문에, 파이썬이 세션을 모두 가져와서 걸러 낼 필요가 없습니다. 실제로 DB에 전달되는 SQL은 다음과 같은 모양입니다.

```
SELECT projects.* FROM projects JOIN project_members ON …
WHERE project_members.user_id = 나 AND projects.deleted_at IS NULL
  AND NOT (EXISTS (SELECT session_id FROM project_sessions
                   WHERE project_sessions.project_id = projects.project_id
                     AND project_sessions.user_id = 나))
ORDER BY created_at DESC, project_id DESC LIMIT 3
```

3. 코드

import (파일 맨 위: 66번과 같고, 이번에 새로 추가한 것은 없습니다)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.models import Project, ProjectMember, ProjectSession, User
from app.schemas.dashboard import (
    DashboardAi,
    DashboardResponse,
    DashboardStorage,
    ReadyProject,
    RecentProject,
    SessionItem,
    StorageByKind,
    UsageCount,
)
```

이 함수는 `select`, `Session`, `Project`, `ProjectMember`, `ProjectSession`, `User`, `ReadyProject`와 상수 `READY_PROJECT_COUNT`(3)를 사용합니다.

함수

```
def get_ready_projects(db: Session, user: User) -> list[ReadyProject]:
    opened = (
        select(ProjectSession.session_id)
        .where(
            ProjectSession.project_id == Project.project_id,
            ProjectSession.user_id == user.user_id,
        )
        .exists()
    )
    projects = db.scalars(
        select(Project)
        .join(ProjectMember, ProjectMember.project_id == Project.project_id)
        .where(ProjectMember.user_id == user.user_id, Project.deleted_at.is_(None), ~opened)
        .order_by(Project.created_at.desc(), Project.project_id.desc())
        .limit(READY_PROJECT_COUNT)
    ).all()
    return [
        ReadyProject(
            project_id=p.project_id,
            title=p.title,
            bpm=p.bpm,
            time_sig_num=p.time_sig_num,
            time_sig_den=p.time_sig_den,
            created_at=p.created_at,
        )
        for p in projects
    ]
```

문법 풀이

import 문법	뜻
(새로 추가한 import 없음)	66번에서 추가한 것을 그대로 사용합니다

식	뜻	예
`select(ProjectSession.session_id).where(…).exists()`	"그런 세션 행이 있다"는 조건을 만듭니다 (EXISTS 서브쿼리)	
`ProjectSession.project_id == Project.project_id`	바깥 쿼리가 지금 보고 있는 **그 행**의 프로젝트와 연결합니다 (상관 서브쿼리)	
`~opened`	조건을 뒤집어서 `NOT EXISTS`로 만듭니다	SQLAlchemy에서는 파이썬의 `not` 대신 `~`를 씁니다
`.order_by(Project.created_at.desc(), …)`	최근에 **만든** 순서로 정렬합니다	66번은 최근에 **수정한** 순서입니다
`.limit(READY_PROJECT_COUNT)`	최대 3개까지만 가져옵니다	

4. 흐름

1. "내 세션이 있다"는 조건을 만듭니다.
2. 내가 멤버이고, 삭제되지 않았으며, 그 조건에 해당하지 **않는** 프로젝트를 최근에 만든 순서로 3개까지 가져옵니다.
3. 가져온 프로젝트를 응답 모양(`ReadyProject`)으로 바꿔서 돌려줍니다.

5. 왜 이렇게

선택	이유
`NOT EXISTS`로 쿼리를 **한 번**만 보냄	프로젝트마다 세션을 따로 묻지 않기 위해서입니다. 시험에서 SELECT가 1번만 나가고 `EXISTS`가 들어 있는 것을 확인했습니다 (템플릿 권장 방식)
내 세션만 확인함	58번 `list_projects`의 `opened`와 기준이 같아야, 목록의 `Not opened` 표시와 배너 내용이 서로 어긋나지 않습니다
기간을 제한하지 않음	7일 전이든 1년 전이든 한 번 열었다면 그 프로젝트는 "열어 본 프로젝트"이지, 시작을 기다리는 프로젝트가 아니기 때문입니다. 최근 7일만 보는 66번과 다른 점입니다
최근에 **만든** 순서	방금 만든 프로젝트가 배너 맨 앞에 나와야 하기 때문입니다 (FR-130)
최대 3개	화면은 1개만 보여 주고 나머지는 개수로 알려 주기 때문입니다 (템플릿)

6. 시험 (`tests/test_dashboard_service.py`: 4개를 추가해서 모두 16개)

시험	확인한 것
프로젝트가 없음	`[]`를 돌려줍니다
범위, 순서, 최대 3개	30일 전에 연 프로젝트, 삭제한 프로젝트, 남의 프로젝트는 빠지고, 다른 멤버만 연 공유 프로젝트는 포함되어 `New 0, Shared, New 1`이 나옵니다
응답 칸	bpm `96.00`, 박자 `7/8`, 생성 시각이 들어 있습니다
쿼리 수	SELECT가 1번만 나가고 `EXISTS`가 들어 있습니다

전체 시험 512개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 68 — `get_dashboard()` (대시보드 응답 조립)

항목	내용
파일	`studionx/backend/app/services/dashboard_service.py`
작업	P1-16 대시보드 API: 마이페이지 전체 응답 만들기 (PRD 01 FR-121, 128, 130, 131, 136)
작성	Claude (AI), 2026-10-12, 사용자 검토
이전 · 다음	67 `get_ready_projects()` · 69 `GET /me/dashboard` 라우터 (다음)

1. 하는 일

63~67번에서 만든 조각들을 **한 번에 묶어서** `DashboardResponse` 하나로 돌려줍니다. 새로 계산하는 값은 없고, 프로필 칸만 이 함수가 직접 채웁니다.

응답 칸	만드는 곳
`user`	이 함수 (닉네임, 이메일, 가입일, 사진 주소)
`storage`	64 `get_storage`
`ai`	65 `get_ai_usage`
`ready_projects`	67 `get_ready_projects`
`recent_projects`	66 `get_recent_projects`

비밀번호 해시나 `role`, `status` 같은 나머지 `User` 칸은 응답에 넣지 않습니다. 칸을 하나씩 골라 넣기 때문에, 나중에 `User`에 칸이 늘어나도 응답에 저절로 섞여 나가지 않습니다.

2. 언제 쓰나

호출하는 곳	화면
69 `GET /me/dashboard` 라우터(예정)	myPage.html 전체 (P1-17)

사진 주소는 P1-08에서 만든 `user_service.profile_image_url`을 그대로 씁니다. 그래서 회원 정보 화면과 마이페이지의 사진 주소가 항상 같은 규칙으로 만들어집니다. 사진이 없으면 `None`이고, 화면은 닉네임 첫 글자로 동그라미를 그립니다.

3. 코드

import (파일 맨 위, 이번에 추가한 것만)

```
from app.schemas.dashboard import (
    ...
    DashboardUser,      # 추가
    ...
```

```
)
from app.services.user_service import profile_image_url   # 추가
```

함수

```
def get_dashboard(db: Session, user: User) -> DashboardResponse:
    return DashboardResponse(
        user=DashboardUser(
            nickname=user.nickname,
            email=user.email,
            created_at=user.created_at,
            profile_image_url=profile_image_url(user),
        ),
        storage=get_storage(user),
        ai=get_ai_usage(db, user),
        ready_projects=get_ready_projects(db, user),
        recent_projects=get_recent_projects(db, user),
    )
```

문법 풀이

식	뜻
`DashboardResponse(user=…, storage=…, …)`	Pydantic 모델을 칸 이름으로 만듭니다. 칸이 빠지거나 타입이 틀리면 이 자리에서 오류가 납니다
`DashboardUser(…)`	`user` 칸의 하위 모양입니다. `User` 객체를 통째로 넘기지 않고 네 칸만 골라 넣습니다
`profile_image_url(user)`	저장 경로 `profiles/3f9c….png`를 화면용 주소 `/api/v1/profile-images/3f9c….png`로 바꿉니다

4. 흐름

1. 프로필 네 칸을 `DashboardUser`로 만듭니다 (사진 주소 변환 포함).
2. `get_storage`, `get_ai_usage`, `get_ready_projects`, `get_recent_projects`를 차례로 부릅니다.
3. 다섯 조각을 `DashboardResponse` 하나로 묶어 돌려줍니다.

5. 왜 이렇게

선택	이유
조립만 하는 함수로 분리	각 조각은 63~67번에서 따로 시험을 마쳤습니다. 이 함수는 "조각을 빠짐없이 올바른 칸에 넣는지"만 확인하면 됩니다
`profile_image_url`을 재사용	사진 주소 규칙(회원 번호를 주소에 넣지 않음)을 한 곳에서만 관리합니다
칸을 하나씩 골라 넣음	해시 등 민감한 값이 응답에 섞일 가능성을 구조적으로 없앱니다 (`to_me_response`와 같은 원칙)
DB 조회를 이 함수에서 직접 하지 않음	조회는 각 `get_*`가 맡고, 쿼리 수도 그쪽에서 이미 줄여 두었습니다

6. 시험 (`tests/test_dashboard_service.py`에 5개 추가, 모두 21개)

실제 MySQL로 확인했습니다. 넣은 행은 시험이 끝나면 되돌립니다.

시험	확인한 것
프로필 블록	닉네임, 이메일, 가입일이 맞고, 사진 주소가 `/api/v1/profile-images/<파일명>`입니다
사진이 없음	`profile_image_url`이 `None`입니다
최상위 칸과 비밀 값	칸이 `user`, `storage`, `ai`, `ready_projects`, `recent_projects` 다섯 개뿐이고, `user` 안에는 네 칸뿐이며, `password_hash`가 응답 어디에도 없습니다
조각 합치기	저장 사용량 42, AI 한도 5/30, 최근 프로젝트 순서(New → Old), 이어 하기 2개가 모두 맞습니다
빈 계정	프로젝트가 없으면 두 목록이 모두 빈 목록입니다

전체 시험 517개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 69 — `GET /me/dashboard` 라우터 (`read_dashboard()`)

항목	내용
파일	`studionx/backend/app/api/v1/me.py`
작업	P1-16 대시보드 API: 마이페이지가 부르는 API 주소 (PRD 01 FR-121, FR-128)
작성	Claude (AI), 2026-10-12, 사용자 검토
이전 · 다음	68 `get_dashboard()` · (P1-16 끝, 사이트 시험은 개요 문서에서)

1. 하는 일

로그인한 회원의 마이페이지 정보를 **요청 한 번으로** 돌려주는 API입니다. 계산은 68번 `get_dashboard`가 모두 하고, 이 라우터는 "로그인한 회원과 DB 세션을 넘기고 결과를 응답으로 내보내는" 일만 합니다.

`GET /api/v1/me/dashboard` + `Authorization: Bearer <Access Token>` → 200 `DashboardResponse`

요청	응답
토큰이 없거나 만료됨	401 `UNAUTHORIZED` ("Please sign in.")
탈퇴하거나 정지된 회원의 토큰	401 `UNAUTHORIZED` ("Please sign in again.")
정상	200, `user`, `storage`, `ai`, `ready_projects`, `recent_projects` 다섯 칸

2. 언제 쓰나

호출하는 곳	화면
`pages/mypage.js`의 `loadDashboard()` (P1-17)	myPage.html을 열 때 한 번

주소에 회원 번호를 받는 칸이 없습니다. 항상 **토큰의 주인 본인**의 대시보드만 나오므로, 번호를 바꿔 남의 정보를 보는 일이 구조적으로 불가능합니다.

3. 코드

import (이번에 추가한 것만)

```
from app.services import dashboard_service
```

`router`, `CurrentUser`, `Db`, `ErrorResponse`, `DashboardResponse`는 템플릿에 이미 있었습니다.

함수

```
@router.get(
    "/dashboard",
    response_model=DashboardResponse,
    responses={401: {"model": ErrorResponse}},
```

```
)
def read_dashboard(user: CurrentUser, db: Db) -> DashboardResponse:
    return dashboard_service.get_dashboard(db, user)
```

문법 풀이

식	뜻
`@router.get("/dashboard", …)`	이 함수를 `GET /me/dashboard`에 연결합니다. 앞의 `/me`는 `router.py`에서 붙입니다
`response_model=DashboardResponse`	돌려주는 값을 이 모양으로 걸러서 JSON으로 만듭니다. 문서(`/docs`)에도 같은 모양이 나옵니다
`responses={401: …}`	`/docs`에 401 응답 모양을 보여 주기 위한 표시입니다. 동작은 바꾸지 않습니다
`user: CurrentUser`	FastAPI가 이 함수보다 **먼저** 06번 `get_current_user`를 실행해서, 검사를 통과한 회원을 넣어 줍니다
`db: Db`	요청마다 DB 세션을 열고, 끝나면 닫아 줍니다

4. 흐름

1. FastAPI가 `Authorization` 헤더를 검사합니다 (05, 06번). 실패하면 401로 응답하고 이 함수는 실행되지 않습니다.
2. 통과하면 회원(`user`)과 DB 세션(`db`)이 들어와 `read_dashboard`가 실행됩니다.
3. `get_dashboard`가 만든 `DashboardResponse`를 그대로 돌려줍니다.

5. 왜 이렇게

선택	이유
라우터에는 계산을 넣지 않음	계산은 서비스(63~68번)에 모아 두었기 때문에, 서비스는 서버를 켜지 않고도 시험할 수 있습니다
`CurrentUser`를 씀 (`OptionalUser`가 아님)	마이페이지는 로그인한 사람만 봅니다. 손님이면 라우터까지 오기 전에 401로 막습니다
요청 한 번으로 모두 돌려줌	마이페이지는 열 때 모든 칸이 필요하므로, 요청을 나누면 화면이 조금씩 채워지며 깜빡입니다
오류 처리를 따로 쓰지 않음	401은 `get_current_user`가, 예상 못 한 오류는 04번 처리기가 같은 JSON 모양으로 응답합니다

6. 시험 (`tests/test_me_routes.py`, 모두 5개)

실제 앱과 실제 MySQL로 요청했습니다 (`dependency_overrides`로 시험용 DB 세션을 넣고, 끝나면 되돌립니다).

시험	확인한 것
주소 연결	`/api/v1/me/dashboard`에 `GET`이 등록되어 있습니다
토큰 없음	401, 오류 코드 `UNAUTHORIZED`
정상 요청	200, 다섯 칸, 이메일, 사진 `null`, 저장 한도 5GB, AI 한도 5/30이 맞고, `password_hash`가 응답 어디에도 없습니다
내 프로젝트만	다른 회원의 프로젝트는 최근 목록과 이어 하기 목록 어디에도 나오지 않습니다

시험	확인한 것
탈퇴한 회원	유효한 토큰이어도 401입니다

전체 시험 521개가 통과했고, `ruff` 검사도 통과했습니다.